

# 管理者画面機能のさらなる発展

ここからは、「打刻モードの統括管理（一元管理）」ができるように管理者画面をさらに発展させていきましょう。

一般ユーザーの画面から切り替えボタンを隠し、最高管理者が裏側のコントロールパネルから「今日のアプリ全体の打刻モード」を一括で切り替えられるように仕組みを整えていきます。

これを実現するために、まずは「管理者用コントロールパネルの全体構成（どう画面やデータを改めるべきか）」の設計図から整理していきましょう。

## 画面・データ構成をどう改めるべきか？

現在は「管理者画面ボタン = アカウント一覧画面」に直結していますが、これを「管理者専用のメニュー画面（ポータル）」を1つ挟むか、あるいは「タブ切り替え」の構造にアップグレードします。

今回は、仕様書にある「お知らせ情報登録」や「月別打刻状況参照出力」といった将来的な管理者機能の拡張にも耐えられるよう、左側にメニュー（あるいは上部にタブ）を配した「管理者専用ダッシュボード（コントロールパネル）」として構成を整える方向でアプローチします。

必要なステップは以下の3つの階層に分かれます。

## データベース（JPA）層の拡張

アプリ全体の「現在の打刻モード」を保存・維持する場所が必要です。H2Database（またはMySQL）に、モードを記録する小さな設定管理用テーブルを作るか、あるいは既存の仕組みを応用します。

- モードの状態定義（例）
  - 0: 勤怠モード（初期状態）
  - 1: 入退室モード
  - 2: 出席退席モード

## バックエンド（Java）APIの新規構築

新しく `SystemConfigController.java`（仮称）のような窓口を作り、以下の2つのAPIを用意します。

- `GET /api/system/mode` : 現在の打刻モードを取得する（ログインしている全ユーザーが、自分の打刻画面を開いた時に裏でこれを読み込んで画面を切り替えます）。
- `PUT /api/system/mode` : 管理者がモードを上書き保存する（**管理者権限チェック必須**）。

## フロントエンド（Vue.js）の画面再構成

管理者画面を「メニュー（タブ）で切り替えられる親画面」に変更します。

- `AdminDashboardView.vue`（新設）：管理者ポータルの器。
  - ↳ タブ1: アカウント一覧・退会管理（今回完成したデータテーブル）
  - ↳ タブ2: 打刻モード一括制御（新設）
  - ↳ タブ3: お知らせ登録（将来用）
  - ↳ タブ4: 月別データ出力（将来用）

## バックエンド（Java）に「モード共有の仕組み」を作る

アプリ全体の現在のモードを管理・返却するAPIの基盤をバックエンドに構築しましょう。

まずはシンプルかつ確実に動かすため、設定情報をメモリ上（JPAエンティティ、または簡易的なサービス内変数）で管理するAPIを作成します。

## SystemConfigController.java の新設

`controller` パッケージ配下に、新しく以下のコントローラーファイルを作成してください。

```
package com.appspace.backend.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/system")
@CrossOrigin(origins = "https://*.app.github.dev") // 環境に合わせて調整してください
public class SystemConfigController {

    // 初期状態は 「0: 勤怠モード」 に設定
    private int currentAttendanceMode = 0;
```

```

/**
 * 現在の打刻モードを取得する（全ユーザーが利用）
 * GET http://localhost:8080/api/system/mode
 */
@GetMapping("/mode")
public ResponseEntity<Map<String, Object>> getAttendanceMode() {
    // Vue側で扱いやすいように JSON オブジェクトの形（{ "mode": 0 }）で返却
    return ResponseEntity.ok(Map.of("mode", currentAttendanceMode));
}

/**
 * 統括管理者が打刻モードを一括変更するエンドポイント
 * PUT http://localhost:8080/api/system/mode
 */
@PutMapping("/mode")
public ResponseEntity<String> updateAttendanceMode(@RequestBody Map<String, Integer> requestBody) {
    try {
        if (!requestBody.containsKey("mode")) {
            return ResponseEntity.badRequest().body("モード値が指定されていません。");
        }

        int newMode = requestBody.get("mode");

        // 0, 1, 2 以外の不正な数値はバリデーションで弾く
        if (newMode < 0 || newMode > 2) {
            return ResponseEntity.badRequest().body("無効なモード値です。");
        }

        // モードを更新
        this.currentAttendanceMode = newMode;
        System.out.println("=== [System] 統括管理者によって打刻モードが変更されました: " + currentAttendanceMode + " ===");

        return ResponseEntity.ok("打刻モードを更新しました。");
    } catch (Exception e) {
        return ResponseEntity.badRequest().body("更新失敗: " + e.getMessage());
    }
}
}

```

## SecurityConfig.java へのURL許可追加

新設した `/api/system/mode` を Spring Security で弾かれないように設定します。

- `GET` のリクエストは、一般ユーザーが打刻画面を開いたときにも自動実行されるため、誰でもアクセスできるようにします。
- `PUT` のリクエストは、後ほど管理者ガードをフロントとバックの両面でかけます。

`SecurityConfig.java` の `.requestMatchers(...)` のリストの中に、以下の一行を優しく追加して、SpringBootを再起動してください。

```
"/api/system/**", // ★システム設定系（打刻モード等）のパスを一括許可リストに追加
```

# フロントエンド（Vue.js）の実装

今回は、今後の機能拡張（お知らせ登録やデータ出力など）を見据えた「タブ切り替え式の管理者ダッシュボード」を構築します。

以下の2つのステップで実装を行います。

1. 既存のデータテーブル（アカウント一覧）を一つの部品（コンポーネント）に切り出す
2. それらを束ねる親画面（管理者ダッシュボード）を新設する

## 既存のアカウント一覧を部品化する（AdminUserList.vue）

現在の `AdminUserListView.vue` の内容を、他の画面から呼び出せる「部品（コンポーネント）」として綺麗に再利用します。

`src/components` フォルダ（なければ `src/views` のままでも動作します）の中に、新しく `AdminUserList.vue` というファイルを作成し、先ほど完成した高機能データテーブルのコードをそのまま貼り付けます。



```
<template>
  <div class="admin-sub-container">
    <div class="control-panel">
      <div class="search-box">
        <label for="searchQuery" class="control-label">アカウント検索</label>
        <input
          id="searchQuery"
          type="text"
          v-model="searchQuery"
          placeholder="名前やIDで検索（半角スペース区切りで複数ワード検索対応）"
          class="input-search"
        />
      </div>

      <div class="per-page-box">
        <label for="perPage" class="control-label">表示件数</label>
        <select id="perPage" v-model="perPage" class="select-per-page">
          <option :value="10">10件</option>
          <option :value="50">50件</option>
          <option :value="100">100件</option>
          <option :value="500">500件</option>
        </select>
      </div>
    </div>

    <div v-if="errorMessage" class="alert alert-danger">{{ errorMessage }}</div>
    <div v-if="successMessage" class="alert alert-success">{{ successMessage }}</div>
  </div>
</template>
```

```
<div v-if="filteredAndSortedUsers.length > 0" class="table-responsive">
  <table class="user-table">
    <thead>
      <tr>
        <th @click="toggleSort('userName')" class="sortable-header">
          ユーザー名 {{ getSortIcon('userName') }}
        </th>
        <th @click="toggleSort('userId')" class="sortable-header">
          ユーザーID (メールアドレス) {{ getSortIcon('userId') }}
        </th>
        <th @click="toggleSort('isAuth')" class="sortable-header">
          権限 {{ getSortIcon('isAuth') }}
        </th>
        <th @click="toggleSort('quitDemand')" class="sortable-header">
          ステータス {{ getSortIcon('quitDemand') }}
        </th>
        <th>操作</th>
      </tr>
    </thead>
```

```
<tbody>
  <tr v-for="user in paginatedUsers" :key="user.accountId" :class="{ 'row-quit-pending': user.quitDemand === 1 }">
    <td><strong>{{ user.userName }}</strong></td>
    <td>{{ user.userId }}</td>
    <td>
      <span :class="['badge', user.isAuth === 1 ? 'badge-admin' : 'badge-general']">
        {{ user.isAuth === 1 ? '統括管理者' : '一般ユーザー' }}
      </span>
    </td>
    <td>
      <span v-if="user.quitDemand === 1" class="badge badge-warning animate-pulse">
        退会申請中
      </span>
      <span v-else class="badge badge-normal">
        正常稼働
      </span>
    </td>
    <td>
      <button
        v-if="user.quitDemand === 1 && user.userId !== 'admin@example.com'"
        @click="handleApproveQuit(user)"
        class="btn-approve"
      >
        退会を承認する
      </button>
      <span v-else-if="user.userId === 'admin@example.com'" class="text-muted-info">-</span>
      <span v-else class="text-muted">-</span>
    </td>
  </tr>
</tbody>
```

```

    </table>
  </div>

  <div v-if="filteredAndSortedUsers.length > 0" class="pagination-panel">
    <div class="pagination-info">
      全 {{ filteredAndSortedUsers.length }} 件中 {{ startIndex + 1 }} ～ {{ endIndex }} 件目を表示
    </div>
    <div class="pagination-buttons">
      <button :disabled="currentPage === 1" @click="currentPage--" class="btn-page">◀ 前へ</button>
      <button
        v-for="page in totalPages"
        :key="page"
        @click="currentPage = page"
        :class="['btn-page-number', { 'active': currentPage === page }]"
      >
        {{ page }}
      </button>
      <button :disabled="currentPage === totalPages" @click="currentPage++" class="btn-page">次へ ▶</button>
    </div>
  </div>

  <div v-else class="empty-box">
    条件に一致するアカウントが存在しません。
  </div>
</div>
</template>

```

```
<script setup>
import { ref, onMounted, computed, watch } from 'vue';
import apiClient from '../api';

const users = ref([]);
const errorMessage = ref('');
const successMessage = ref('');

const searchQuery = ref('');
const perPage = ref(10);
const currentPage = ref(1);
const sortKey = ref('quitDemand');
const sortOrder = ref('desc');

const fetchUserList = async () => {
  try {
    const response = await apiClient.get('/users/admin/list');
    users.value = response.data;
  } catch (error) {
    errorMessage.value = 'アカウント一覧の取得に失敗しました。';
  }
};

onMounted(() => {
  fetchUserList();
});
```

```

const filteredAndSortedUsers = computed(() => {
  let result = [...users.value];
  if (searchQuery.value.trim()) {
    const keywords = searchQuery.value.replace(/ /g, ' ').toLowerCase().split(' ').filter(w => w);
    result = result.filter(user => {
      const targetText = `${user.userName} ${user.userId}`.toLowerCase();
      return keywords.every(keyword => targetText.includes(keyword));
    });
  }
  result.sort((a, b) => {
    let modifier = sortOrder.value === 'desc' ? -1 : 1;
    let valA = a[sortKey.value];
    let valB = b[sortKey.value];
    if (typeof valA === 'string') valA = valA.toLowerCase();
    if (typeof valB === 'string') valB = valB.toLowerCase();
    if (valA < valB) return -1 * modifier;
    if (valA > valB) return 1 * modifier;
    return 0;
  });
  return result;
});

const totalPages = computed(() => Math.ceil(filteredAndSortedUsers.value.length / perPage.value) || 1);
const startIndex = computed(() => (currentPage.value - 1) * perPage.value);
const endIndex = computed(() => {
  const end = startIndex.value + perPage.value;
  return end > filteredAndSortedUsers.value.length ? filteredAndSortedUsers.value.length : end;
});
const paginatedUsers = computed(() => filteredAndSortedUsers.value.slice(startIndex.value, endIndex.value));

```

```
watch([searchQuery, perPage], () => { currentPage.value = 1; });

const toggleSort = (key) => {
  if (sortKey.value === key) {
    sortOrder.value = sortOrder.value === 'asc' ? 'desc' : 'asc';
  } else {
    sortKey.value = key;
    sortOrder.value = 'asc';
  }
};

const getSortIcon = (key) => {
  if (sortKey.value !== key) return '↕';
  return sortOrder.value === 'asc' ? '▲' : '▼';
};

const handleApproveQuit = async (targetUser) => {
  if (!confirm(`本当に「${targetUser.userName}」さんの退会申請を承認しますか？\nこの操作を行うと、アカウント情報は完全に物理削除されます。`)) return;
  try {
    await apiClient.delete(`/users/approve-quit/${targetUser.accountId}`);
    successMessage.value = `「${targetUser.userName}」さんの退会承認が完了しました。`;
    await fetchUserList();
  } catch (error) {
    errorMessage.value = '承認処理に失敗しました。';
  }
};
</script>
```

```
<style scoped>
/* スタイルは前回同様（親から切り離したため幅いっぱいに広がるよう調整） */
.control-panel {
  display: flex;
  gap: 20px;
  background: #ffffff;
  padding: 15px 20px;
  border-radius: 8px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.05);
  margin-bottom: 15px;
  align-items: flex-end;
}
.search-box { flex: 1; }
.per-page-box { width: 120px; }
.control-label { display: block; font-size: 0.85rem; font-weight: bold; color: #495057; margin-bottom: 5px; }
.input-search, .select-per-page {
  width: 100%;
  padding: 8px 12px;
  border: 1px solid #ced4da;
  border-radius: 4px;
  font-size: 0.9rem;
  box-sizing: border-box;
}
.sortable-header { cursor: pointer; user-select: none; }
.sortable-header:hover { background-color: #e9ecef !important; }
.table-responsive { background: #ffffff; border-radius: 8px; box-shadow: 0 4px 12px rgba(0,0,0,0.08); overflow: hidden; }
.user-table { width: 100%; border-collapse: collapse; text-align: left; }
.user-table th, .user-table td { padding: 15px 20px; border-bottom: 1px solid #dee2e6; }
.user-table th { background-color: #f8f9fa; color: #495057; font-weight: bold; }
.row-quit-pending { background-color: #fffdf5; }
.badge { display: inline-block; padding: 5px 10px; font-size: 0.8rem; font-weight: bold; border-radius: 20px; }
.badge-admin { background-color: #e3f2fd; color: #0d47a1; }
.badge-general { background-color: #e8f5e9; color: #1b5e20; }
.badge-normal { background-color: #f1f3f5; color: #6c757d; }
.badge-warning { background-color: #fff3cd; color: #856404; border: 1px solid #ffeeba; }
```



```

@keyframes pulse { 0% { opacity: 1; } 50% { opacity: 0.6; } 100% { opacity: 1; } }
.animate-pulse { animation: pulse 2s infinite; }
.btn-approve {
  background-color: #dc3545;
  color: white;
  border: none;
  padding: 8px 14px;
  border-radius: 4px;
  font-size: 0.85rem;
  font-weight: bold;
  cursor: pointer;
}
.btn-approve:hover { background-color: #bd2130; }
.text-muted { color: #ced4da; }
.text-muted-info { color: #adb5bd; font-size: 0.85rem; font-style: italic; }
.pagination-panel {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 20px;
  background: #ffffff;
  padding: 12px 20px;
  border-radius: 8px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.05);
}
.pagination-info { font-size: 0.9rem; color: #495057; }
.pagination-buttons { display: flex; gap: 5px; }
.btn-page, .btn-page-number {
  padding: 6px 12px;
  background-color: #ffffff;
  border: 1px solid #ced4da;
  border-radius: 4px;
  font-size: 0.85rem;
  cursor: pointer;
}
.btn-page:disabled { color: #ced4da; cursor: not-allowed; }
.btn-page-number.active { background-color: #007bff; color: white; border-color: #007bff; font-weight: bold; }
.alert { padding: 15px; margin-bottom: 20px; border-radius: 4px; font-weight: bold; }
.alert-danger { background-color: #f8d7da; color: #721c24; border-left: 5px solid #dc3545; }
.alert-success { background-color: #d4edda; color: #155724; border-left: 5px solid #28a745; }
.empty-box { text-align: center; padding: 40px; color: #6c757d; }
</style>

```

## メインの管理者ポータル画面を作る (AdminUserListView.vue)

ルーターに登録されている `AdminUserListView.vue` を丸ごと以下のコードに置き換えます。

この画面が「親」となり、上部のタブメニューで「アカウント一覧（部品）」と新設する「打刻モード管理」を瞬時に切り替えられるようにします。

```
<template>
  <div class="admin-container">
    <div class="admin-header-box">
      <h2>統括管理者用 コントロールパネル</h2>
      <p class="admin-subtitle">システムの全体管理、モード一括変更、およびアカウント制御を行えます。</p>
    </div>

    <div class="tab-menu">
      <button
        :class="['tab-btn', { active: currentTab === 'users' }]"
        @click="currentTab = 'users'"
      >
        アカウント一覧・退会管理
      </button>
```

```

<button
  :class="['tab-btn', { active: currentTab === 'mode' }]"
  @click="currentTab = 'mode'"
>
  打刻モード一括制御
</button>
</div>
<div class="tab-content">

  <div v-if="currentTab === 'users'">
    <AdminUserList />
  </div>

  <div v-if="currentTab === 'mode'" class="mode-control-box">
    <h3>現在のアプリケーション打刻モード設定</h3>
    <p class="section-desc">ここでモードを切り替えると、アプリを利用する全ユーザーの打刻画面が即座に統一されます。</p>

    <div v-if="modeSuccessMessage" class="alert alert-success">{{ modeSuccessMessage }}</div>
    <div v-if="modeErrorMessage" class="alert alert-danger">{{ modeErrorMessage }}</div>

    <div class="mode-cards-container">
      <div
        :class="['mode-card', { 'active-card': systemMode === 0 }]"
        @click="saveSystemMode(0)"
      >
        <div class="card-icon"><img alt="Briefcase icon" data-bbox="298 725 315 745"/></div>
        <h4>勤怠モード</h4>
        <p>「出勤」「退勤」「休憩開始」「休憩終了」を管理する標準的なモードです。</p>
        <span class="status-indicator">{{ systemMode === 0 ? '稼働中' : '選択する' }}</span>
      </div>
    </div>
  </div>

```

```
<div
  :class="['mode-card', { 'active-card': systemMode === 1 }]"
  @click="saveSystemMode(1)"
>
  <div class="card-icon">📅</div>
  <h4>入退室モード</h4>
  <p>オフィスのセキュリティや「入室」「退室」のログ選定に特化したモードです。</p>
  <span class="status-indicator">{{ systemMode === 1 ? '稼働中' : '選択する' }}</span>
</div>

<div
  :class="['mode-card', { 'active-card': systemMode === 2 }]"
  @click="saveSystemMode(2)"
>
  <div class="card-icon">🗓️</div>
  <h4>出席退席モード</h4>
  <p>講義やイベント、集会などの「出席」「退席」をシンプルに記録するモードです。</p>
  <span class="status-indicator">{{ systemMode === 2 ? '稼働中' : '選択する' }}</span>
</div>
</div>
</div>
</template>
```

```
<script setup>
import { ref, onMounted } from 'vue';
import { useRouter } from 'vue-router';
import apiClient from '../api';
// Step1で作成したコンポーネントをインポート
import AdminUserList from '../components/AdminUserList.vue';

const router = useRouter();

// 現在開いているタブ管理 ('users' または 'mode')
const currentTab = ref('users');

// 打刻モードの状態管理 (0:勤怠, 1:入退室, 2:出席退席)
const systemMode = ref(0);

const modeSuccessMessage = ref('');
const modeErrorMessage = ref('');

// 1. バックエンドから現在のモードを取得する関数
const fetchCurrentMode = async () => {
  try {
    // Java側で作成した GET /api/system/mode を呼び出す
    const response = await apiClient.get('/system/mode');
    systemMode.value = response.data.mode;
  } catch (error) {
    console.error(error);
    modeErrorMessage.value = '現在の打刻モードの取得に失敗しました。';
  }
};
```

```

// 2. 統括管理者がモードを更新・保存する関数
const saveSystemMode = async (targetMode) => {
  modeSuccessMessage.value = '';
  modeErrorMessage.value = '';

  try {
    // Java側で作成した PUT /api/system/mode を呼び出す
    await apiClient.put('/system/mode', { mode: targetMode });

    // リアクティブ変数を更新
    systemMode.value = targetMode;
    modeSuccessMessage.value = 'アプリケーション全体の打刻モードを正常に一括更新しました！';
  } catch (error) {
    modeErrorMessage.value = 'モードの更新中に通信エラーが発生しました。';
  }
};

// 画面表示時の管理者セキュリティガード
onMounted(() => {
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    if (user.isAuth !== 1) {
      alert('この画面は管理者専用です。');
      router.push('/dashboard');
      return;
    }
    // 管理者であれば、現在の打刻モードを初期ロード
    fetchCurrentMode();
  } else {
    router.push('/login');
  }
});
</script>

```

```
<style scoped>
.admin-container {
  max-width: 1000px;
  margin: 30px auto;
  padding: 20px;
}
.admin-header-box {
  background: #ffffff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.05);
  margin-bottom: 25px;
  border-left: 5px solid #dc3545;
}
h2 { margin: 0 0 5px 0; color: #333; }
.admin-subtitle { margin: 0; color: #6c757d; font-size: 0.95rem; }

/* 💡 タブメニューのスタイル */
.tab-menu {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
  border-bottom: 2px solid #dee2e6;
  padding-bottom: 10px;
}
.tab-btn {
  padding: 10px 20px;
  font-size: 1rem;
  font-weight: bold;
  background: none;
  border: none;
  color: #6c757d;
  cursor: pointer;
  transition: all 0.2s;
  border-radius: 4px;
}
.tab-btn:hover {
  background-color: #f8f9fa;
  color: #495057;
}
.tab-btn.active {
  background-color: #dc3545;
  color: white;
  box-shadow: 0 2px 6px rgba(220, 53, 69, 0.3);
}
```

```

/* モード制御コンテンツのスタイル */
.mode-control-box {
  background: #ffffff;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
}
h3 { margin-top: 0; color: #333; }
.section-desc { color: #6c757d; font-size: 0.95rem; margin-bottom: 25px; }

/* カード配置エリア */
.mode-cards-container {
  display: flex;
  gap: 20px;
  margin-top: 20px;
}
.mode-card {
  flex: 1;
  border: 2px solid #e9ecef;
  border-radius: 8px;
  padding: 25px 20px;
  text-align: center;
  cursor: pointer;
  transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
  background: #fff;
  position: relative;
}
.mode-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 6px 15px rgba(0,0,0,0.1);
  border-color: #ced4da;
}
.card-icon { font-size: 3rem; margin-bottom: 15px; }
.mode-card h4 { margin: 0 0 10px 0; font-size: 1.2rem; color: #333; }
.mode-card p { font-size: 0.88rem; color: #6c757d; line-height: 1.5; margin-bottom: 20px; }

.status-indicator {
  display: inline-block;
  padding: 6px 16px;
  font-size: 0.85rem;
  font-weight: bold;
  border-radius: 20px;
  background-color: #f1f3f5;
  color: #6c757d;
  transition: all 0.2s;
}

/* 現在アクティブ（稼働中）なモードカードのスタイル変更 */
.active-card {
  border-color: #28a745 !important; /* 稼働中は安心のグリーン */
  background-color: #f4fbf6;
  box-shadow: 0 4px 12px rgba(40, 167, 69, 0.15);
}
.active-card .status-indicator {
  background-color: #28a745;
  color: white;
}

/* メッセージ用 */
.alert { padding: 15px; margin-bottom: 20px; border-radius: 4px; font-weight: bold; }
.alert-danger { background-color: #f8d7da; color: #721c24; border-left: 5px solid #dc3545; }
.alert-success { background-color: #d4edda; color: #155724; border-left: 5px solid #28a745; }
</style>

```



# 動作確認のシナリオ

ソースコードを保存し、ブラウザで管理者画面（`/admin`）を開いてみましょう！

## 1. タブの切り替えテスト

画面上部に「アカウント一覧」と「打刻モード一括制御」のボタンが並んでいるのを確認します。  
クリックすると、画面の下半分が滑らかに切り替わるかチェックします。

## 2. 初期状態の確認

「打刻モード一括制御」を開いた時、Java側で定義した初期値に従って「勤怠モード」のカードが緑枠（稼働中）に輝いていることを確認します。

## 3. モード変更の疎通テスト

「入退室モード」または「出席退席モード」のカードをどれかクリックしてみてください。  
画面上に「アプリケーション全体の打刻モードを正常に一括更新しました！」とサッと表示され、  
クリックしたカードが即座に「稼働中（緑色）」に切り替われば大成功です！

このとき、Java側のコンソールにも `=== [System] 統括管理者によって打刻モードが変更されました：○`  
`===` とログが流れます。

## 打刻画面（ユーザー側）との連動

これまでは一般ユーザーの画面側にすべてのボタン（出勤・退勤・入室・退室・出席・退席など）が並んでおり、手動で切り替えるつくりになっていました。

今後は、打刻画面が開かれた瞬間に、今回作ったAPI（`GET /api/system/mode`）を裏で呼び出し、その数値によって**表示するボタンを自動で切り替える**つくりにします。

`AttendanceBoard.vue` を次のように書き換えてください。

```
<template>
  <div class="attendance-board">

    <div class="attendance-header-box">
      <h3>ようこそ、{{ userName }} さん</h3>
    </div>
```

```
<div class="status-display">
  <p>現在の状態 :
    <span :class="status === 'CLOCKED_IN' ? 'status-in' : 'status-out'">
      {{ status === 'CLOCKED_IN' ? currentLabels.inActive : currentLabels.outActive }}
    </span>
  </p>
</div>

<div class="punch-actions">
  <button
    v-if="status === 'CLOCKED_OUT'"
    @click="punch('CLOCK_IN')"
    class="btn btn-in"
  >
    {{ currentLabels.inAction }}する
  </button>

  <button
    v-else
    @click="punch('CLOCK_OUT')"
    class="btn btn-out"
  >
    {{ currentLabels.outAction }}する
  </button>
</div>

<div class="system-mode-indicator">
  <span class="indicator-tag">アプリケーション稼働モード : </span>
  <strong class="indicator-text">
    {{ currentMode === 'attendance' ? '勤怠モード' : currentMode === 'room' ? '入退室モード' : '出席退席モード' }}
  </strong>
</div>
```

```

<hr class="divider">

<div class="history-section">
  <h3>最近の履歴（直近10件）</h3>
  <ul class="history-list">
    <li v-for="(entry, index) in pairedHistory" :key="index" class="history-item">
      <span class="date">{{ entry.date }}</span>
      <span class="time">Entry: {{ entry.inTime }}</span>
      <span class="separator"> ~ </span>
      <span class="time">Exit: {{ entry.outTime }}</span>
    </li>
  </ul>
</div>

</div>
</template>

<script setup>
import { ref, onMounted, computed } from 'vue';
import apiClient from '../api';

const history = ref([]);
const loggedInAccountId = ref('');
const userName = ref('');

// 変更点：初期値は 'attendance' とし、文字列で管理する構造を維持します
const currentMode = ref('attendance');
const attendanceHistory = ref([]);

// モードの数値(0, 1, 2)と、既存の文字列キー('attendance', 'room', 'session')をマッピングする辞書
const modeMapping = {
  0: 'attendance', // 勤怠モード
  1: 'room',       // 入退室モード
  2: 'session'     // 出席退席モード
};

```

```
// 追加：バックエンドから最新の一括打刻モードを取得する関数
const fetchSystemMode = async () => {
  try {
    // 先ほどJava側で作成した GET /api/system/mode を呼び出す
    const response = await apiClient.get('/system/mode');
    const modeNumber = response.data.mode; // 0, 1, 2 が返ってくる

    // 数値を対応する文字列キーに変換して、currentModeに格納
    currentMode.value = modeMapping[modeNumber] || 'attendance';
    console.log(`[System] 最新の打刻モードを自動適用しました: ${currentMode.value}`);
  } catch (error) {
    console.error('打刻モードの取得に失敗しました。デフォルト（勤怠）で動作します:', error);
  }
};

// バックエンドから打刻履歴を取得する関数
const fetchAttendanceHistory = async (id) => {
  if (!id) return;
  try {
    const response = await apiClient.get(`/attendance/history/${id}`);
    attendanceHistory.value = response.data;
  } catch (error) {
    console.error('打刻履歴の取得に失敗しました:', error);
  }
};
```

// モードに応じた各種ラベルの定義（既存の定義をそのまま活用）

```
const labelSettings = {  
  attendance: { inActive: '勤務中', outActive: '未出勤', inAction: '出勤', outAction: '退勤' },  
  room:      { inActive: '入室中', outActive: '退室済', inAction: '入室', outAction: '退室' },  
  session:   { inActive: '出席中', outActive: '退席中', inAction: '出席', outAction: '退席' }  
};
```

// 現在選択されているモードのラベル群を返す

```
const currentLabels = computed(() => {  
  return labelSettings[currentMode.value] || labelSettings.attendance;  
});
```

```
const status = ref('CLOCKED_OUT');
```

// ステータス取得API

```
const fetchStatus = async () => {  
  if (!loggedInAccountId.value) return;  
  try {  
    const response = await apiClient.get(`/attendance/status?accountId=${loggedInAccountId.value}`);  
    status.value = response.data;  
  } catch (error) {  
    console.error('ステータス取得失敗', error);  
  }  
};
```

```
// 履歴取得
const fetchHistory = async () => {
  try {
    const response = await apiClient.get(`/attendance/history/${loggedInAccountId.value}`);
    history.value = response.data;
  } catch (error) {
    console.error('履歴取得失敗', error);
  }
};

// 履歴をペアリングして直近10件分を返すロジック
const pairedHistory = computed(() => {
  if (!history.value || !Array.isArray(history.value)) return [];

  const result = [];
  const sortedHistory = [...history.value].reverse();

  for (let i = 0; i < sortedHistory.length; i++) {
    const record = sortedHistory[i];

    if (record.type === 'clock-in') {
      const nextRecord = sortedHistory[i + 1];
      if (nextRecord && nextRecord.type === 'clock-out') {
        result.push({
          date: formatDate(record.createdAt),
          inTime: formatTime(record.createdAt),
          outTime: formatTime(nextRecord.createdAt)
        });
        i++;
      } else {
        result.push({
          date: formatDate(record.createdAt),
          inTime: formatTime(record.createdAt),
          outTime: '-'
        });
      }
    }
  }
  return result.reverse().slice(0, 10);
});
```

```

const emit = defineEmits(['refresh-history']);

// 打刻処理
const punch = async (type) => {
  try {
    await apiClient.post('/attendance/punch', {
      accountId: loggedInAccountId.value,
      type: type
    });

    await fetchStatus();
    await fetchHistory();
    emit('refresh-history');

    fetchAttendanceHistory(loggedInAccountId.value);
  } catch (error) {
    console.error('打刻エラー:', error);
    alert('打刻に失敗しました。');
  }
};

const formatDate = (dateStr) => new Date(dateStr).toLocaleDateString('ja-JP');
const formatTime = (dateStr) => new Date(dateStr).toLocaleTimeString('ja-JP', { hour: '2-digit', minute: '2-digit', second: '2-digit' });

// 💡 画面立ち上げ時の処理をブラッシュアップ
onMounted(async () => {
  // 1. まずは管理者が指定した「最新のモード」を裏でサッと取得
  await fetchSystemMode();

  // 2. ユーザー情報の復元と、ステータス・履歴の読み込み
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    loggedInAccountId.value = user.accountId;
    userName.value = user.userName;

    // ユーザーに紐づくデータをロード
    fetchStatus();
    fetchHistory();
    fetchAttendanceHistory(loggedInAccountId.value);
  } else {
    console.error("ユーザー情報が見つかりません。");
  }
});
</script>

```



```
<style scoped>
.attendance-board { text-align: center; margin-top: 50px; max-width: 600px; margin-left: auto; margin-right: auto; padding: 20px; }
.attendance-header-box { margin-bottom: 20px; }
.status-display { margin-bottom: 25px; font-size: 1.1rem; font-weight: bold; }
.status-in { background-color: #e3f2fd; color: #1976d2; padding: 6px 16px; border-radius: 20px; }
.status-out { background-color: #f5f5f5; color: #616161; padding: 6px 16px; border-radius: 20px; }

.punch-actions { margin-bottom: 30px; }
.btn-in {
  background-color: #4caf50;
  color: white;
  padding: 18px 40px;
  font-size: 1.4rem;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
  box-shadow: 0 4px 6px rgba(76,175,80,0.2);
  transition: background-color 0.2s;
}
.btn-in:hover { background-color: #43a047; }
.btn-out {
  background-color: #f44336;
  color: white;
  padding: 18px 40px;
  font-size: 1.4rem;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
  box-shadow: 0 4px 6px rgba(244,67,54,0.2);
  transition: background-color 0.2s;
}
.btn-out:hover { background-color: #e53935; }
```

```
/* 追加：現在のシステム稼働モード案内用インジケータの装飾 */
.system-mode-indicator {
  background-color: #f8f9fa;
  border: 1px solid #e9ecef;
  border-radius: 6px;
  padding: 10px 15px;
  display: inline-flex;
  align-items: center;
  gap: 10px;
  font-size: 0.9rem;
  margin-top: 10px;
  margin-bottom: 10px;
}
.indicator-tag {
  color: #6c757d;
  font-size: 0.8rem;
  background-color: #e9ecef;
  padding: 2px 8px;
  border-radius: 4px;
}
.indicator-text {
  color: #212529;
}

.divider { margin: 30px 0; border: none; border-top: 1px solid #e9ecef; }
.history-section h3 { color: #495057; margin-bottom: 15px; }
.history-list { list-style: none; padding: 0; margin: 0; }
.history-item {
  display: flex;
  justify-content: center;
  gap: 15px;
  padding: 10px 0;
  border-bottom: 1px solid #f1f3f5;
  font-family: monospace;
  font-size: 0.95rem;
  color: #495057;
}
.separator { color: #ced4da; }
</style>
```

## これによって実現するシナリオ

1. 管理者がコントロールパネルで「入退室モード」に設定して保存する。
2. 一般ユーザーがスマホやPCでアプリの打刻画面を開くと、裏側で設定値 **1** が読み込まれる。
3. ユーザーの画面には、余計なボタンが一切消え、「入室」「退室」の2つのボタンだけがスマートに表示される。

この仕組みができあがると、現場の状況（「今日は社内イベントだから出席退席モードにしよう」「通常業務だから勤怠モードにしよう」など）に合わせて、システム全体を管理者が完全に手元でコントロールできるようになります。